MAXX MapUI: classic map with current-world inputs

Prepared October 26, 2026. Partners: David and Shek. Demonstration: October 27. Git target: **MapUI**, replacing the earlier human-only submission target for this version.

1. What you are building

A software robot follows a route on current downloaded OSM campus geometry. The backend periodically reads real published campus project and event sources. It turns those records into map overlays and carefully labeled planning inputs. The frontend shows robot movement, ETA, data provenance, condition age, route availability and switches.

The customer can pan and zoom and view ETA on desktop or a responsive phone layout. Position display reads every 10 seconds, or every 1 second when selected. Both choices show the same shared software robot. Refresh speed does not turn a model position into GPS.

Component	Actual source and status	How it affects the map
Paths/buildings	OSM map API snapshot, retrieved October 26; 145 real building outlines	Geometry and graph; optional online background tiles
Construction projects	Public feature layer used by Mason's official Construction page	Published project polygons; optional conservative avoidance of Construction-phase areas
Events	Verified Communication Department iCalendar feed for event 18428	Actual time/location record; low-confidence crowd prediction only during that window
Specific walkway closures	Actual operator reports, with reporter/time and explicit clearance	Hard exclusions for affected graph segments
Observed crowds	Actual operator reports, intensity/confidence/radius and five-minute expiry	Weighted spatial field and edge travel costs
Robot location	Your shared software robot engine	Computed movement along its planned graph route; no physical GPS claim
Delivery stops	Six planned graph targets; optional separately verified actual places	Software test destinations; not asserted real customer orders

The public event integration is intentionally narrow: one verified event feed, not the entire university calendar. It currently describes tomorrow's Communication Career Forum at the Johnson Center, 8:30 AM-12:30 PM Eastern. No actual crowd counter, full class timetable, or complete real-time campus closure service was verified. The source panel says this explicitly.

Shek owns the software-robot/position-feed integration opportunity and is credited for the route-as-robot concept. David owns the map interaction, ETA presentation and mobile acceptance review.

2. Start the local working map

Extract the ZIP. Open `MAXX_MapUI_Live_Kit` as a project in PyCharm. Select an existing Python 3.10+ interpreter. No third-party Python packages are needed to RUN the supplied app; the PDF is already generated.

Open Terminal at the bottom. Commands go into Terminal; Python/HTML/CSS/JavaScript source goes into editor files. Enter these commands one at a time and press Enter:

```
python --version
dir
python server.py
```

`dir` should show `server.py`, `world.py`, `public_sources.py`, `data` and `web`. If Python is not recognized, use `py -3`. Stop the old port-8000 server with Ctrl+C in its own terminal before starting this one.

Open the updated classic page. The canonical path is also supported at `http://127.0.0.1:8000/html/web/public/dave-campus-map/gis-layers.html`. If localhost still shows the old saved-route page, use this explicit IPv4 address to reach the kit server. Keep the server terminal open. The original repository HTML is not overwritten: this Python server maps that URL to the supplied `web/world.html`.

If port 8000 remains occupied, use `python server.py --port 8004` and replace the URL's port with 8004. Do not kill an unknown process. A plain `python -m http.server` cannot provide this kit's backend APIs.

Wait for source connectivity. The server starts a background thread that refreshes actual public sources, initially on startup and then every 15 minutes (retrying after about a minute on a failed refresh). The UI's 1/10-second polling reads the robot and current source status; it does not repeatedly download source maps or events every second.

3. Backend-to-frontend data path

File	Purpose
`server.py`	Same-origin APIs, mode selection, static files and operator authorization
`engine.py`	OSM graph, Dijkstra, time weights and shared software movement
`map_features.py`	Named OSM building-derived delivery targets and actual access-tag overlays
`world.py`	Combines public inputs and actual reports for the software robot
`public_sources.py`	Public HTTP requests, validation, normalization, source cache and refresh thread
`live_state.py`	Actual staff reports, verified places, stale data and persistence
`crowd_model.py`	Common weight/freshness/distance/edge-exposure model
`web/world.html`, `web/world.js`	Main software-robot UI and source-status panel
`web/live.html`, `web/live.js`	Optional actual staff-report interface
`web/heat-field.js`, `web/heat-layer.js`	Matching browser crowd-field and heat renderer
`simulation_server.py`, `web/index.html`, `web/app.js`	Preserved offline/synthetic demonstration
`refresh_osm.py`, `refresh_sources.py`	Explicit map refresh and source diagnosis

The browser GETs `/api/config` and `/api/layers`, then `/api/state`. A state contains the computed robot position and routes, source statuses, published project polygons, event records, heat points and node scores. The browser mutates existing map layers rather than recreating the map, preserving pan/zoom and selected visibility. Visual Update 2 restores the old map-first layout: route status, heading, instruction, large map, counts/key, then new controls below. OpenStreetMap background tiles start enabled. Main checkboxes and route switches stay open at top-right; advanced heat and source overlays use the bottom-left control.

Backend conditions are authoritative for route costs. A visibility checkbox only hides a layer; it does not disable a safety exclusion or crowd cost. GeoJSON uses longitude first; Leaflet helpers use latitude first. Keep conversion at the rendering boundary.

4. What makes the inputs real

The source registry includes the official discovery page, underlying endpoint and observed result. Mason's Construction page directly links its ArcGIS experience; that experience identifies the Project_Polygons feature service. We queried its public read-only endpoint successfully. Only public map properties needed for routing/display are retained; contact, budget and unrelated details are excluded.

The project's schema has phase and footprint, but no assertion that every footprint is a currently closed sidewalk. Consequently the default shows project areas without mislabeling them. The optional avoidance setting conservatively removes segments sampled inside **Construction** phase polygons; Planning, Complete and Closeout are not automatically called blocked. This is an engineering policy, not an authoritative closure list.

The event `.ics` endpoint was retrieved successfully. The parser reads UID, summary, location and actual start/end. The feed declares Eastern time; the parser applies that declared zone rather than the laptop's local zone. Unsupported recurrence/timezone cases fail validation instead of silently guessing. Cancelled events are omitted.

The Johnson Center event location is matched to the named OSM building. Its crowd anchor is a building-derived center, not a surveyed entrance or measured crowd location. An 80-meter influence radius, intensity 0.5 and confidence 0.3 are transparent tunable heuristics. Neither attendance nor student count comes from the event feed. Outside the event's time window, its crowd influence is zero.

The generally advertised Today@Mason Trumba JSON/RSS endpoints returned errors during verification. They were not substituted with fictional events. A full calendar/registrar export still needs a verified accessible feed and appropriate permissions. Add future approved sources by normalizing them into the same validated event/crowd schema; no private individual schedules are included.

5. Source age, failures and replanning

`data/public-cache` preserves each last successful source response with its URL and fetch time. This is different from the publisher's update/observation time. The construction source may contain estimates like “Estimated 2026”; frequent polling cannot make such an estimate precise.

Sources are fresh for at most one hour and must have a successful latest request. An error remains visible. Last-known project polygons are retained for conservative exclusions, not silently removed to reopen a potential obstruction. Start in current-world mode is disabled if either verified public source is unavailable/stale. Use `START_SIMULATION.cmd` for a deliberately separate offline demonstration.

When meaningful source/report conditions change, the software trip pauses and resets to base for a newly planned review. The UI states that reset. It does not silently animate through a new closure or pretend the software position was observed. After review, click Start. Event time-window transitions also trigger re-evaluation; merely fetching identical normalized source data does not reset the trip.

The return plan is checked against current known conditions. If conditions change while the software robot is moving, the trip resets for review. Predicting unknown future closures, continuous physical obstacle avoidance and certified robot route safety are not claimed.

6. Movement, ETA and six round trips

Dijkstra minimizes nonnegative estimated travel seconds. The graph joins only shared source node IDs; visually crossing paths are not automatically connected. The pedestrian demonstration policy excludes steps, road classes, private/no-access footways and polygon areas. The display also includes eight actual OSM access-tag restrictions, with exact `access=delivery` or `access=official` values in popups. A red restricted-entry symbol is shared by map markers and the legend. These limitations are not automatically described as construction closures. `motor_vehicle=no` plus `foot=yes` stays a walkable displayed path, with the actual tags exposed in its tooltip. It is not a complete real-robot permission model: gates, slope, width, barriers, indoor access and campus authorization still require review.

The base is a public OSM footprint/path coordinate at Southside Dining. Six distinct software delivery targets are derived from OSM footprints for Hanover Hall, Brunswick Hall, Essex Hall, Grayson Hall, Carroll Hall and Dominion Hall. The nearest connected path node is selected against actual footprint vertices, with a 40-meter maximum reviewed snap. In this bundled snapshot all seven source anchors coincide with path nodes. Source building IDs and snap offsets are exposed in configuration. These are not surveyed entrances, six real orders, or a ranking of the largest dorms.

The software clock determines distance along route segments; UI polling never adds a movement step. Multiple phones therefore see the same robot, regardless of individual polling cadence. Pause freezes the model clock. Outbound ETA is time to delivery; returning ETA is time to base. No valid complete trip means unavailable, no movement and no invented straight line.

At 1x, ETA is modeled travel time. At 10x, it is explicitly accelerated presentation time. Speed assumptions are demonstration parameters, not a blanket 15 mph or a legal/campus operating limit. No 20-minute service guarantee or delivery dwell is enforced in this version.

Ordinary returns exactly retrace the outward route under the current known snapshot. The separate synthetic mode retains forecast-return blockage and alternative-return scenarios for reproducible testing. Actual newly reported closures affect the current-world route and trigger the review/reset policy.

7. Weighted crowds without double-counting

Each point records source, data mode, location, intensity (`weight`), confidence, freshness half-life, radius and active start/end. Source modes distinguish actual operator observations from event-based predictions and synthetic exercise data.

Effective weight is intensity x confidence x `2^(-age_minutes/half_life_minutes)`, only within the active window. Distance contribution falls quadratically to zero at the radius. Overlapping contributions combine as `1-product(1-contribution)`, bounded to 0-1.

Walkways are sampled at no more than 10-meter spacing. Edge exposure is the length-weighted mean field value; edge duration is length/base-speed x `(1+3*exposure)`. No extra node cost is added. Node scores are an optional explanation overlay. Duplicate observation IDs update their existing record rather than adding the same observation twice. Different-source duplicate IDs are rejected.

The heat renderer evaluates the same field at a 12-pixel grid and uses a fixed color scale. It is a sampled relative crowd index, not people per square meter. A schedule indicates a possible crowd; only a real count/sensor could supply measured attendance.

8. Actual staff reports and local persistence

Open the report panel. Enter the operator key printed in the terminal, your reporter/source name, and choose a report type. Click the actual location. Confirm you actually checked the observation/place, then submit.

- **Location:** describes STAFF-TEST-01, a field participant. It is a manual map report, not hardware GPS. It never moves by timer. After 30 seconds without another actual observation, its ETA becomes unavailable.

- **Place:** verify a base and up to six distinct real destinations; snaps over 30 meters are rejected. These verified places are separate from the software robot's planned demonstration stops.

- **Closure:** clicks choose a nearby graph segment. It remains blocked until explicit clearance, even after its one-hour review deadline. These reports influence the current-world software robot.

- **Crowd:** enter a stable observation ID, name, intensity, confidence and radius. Reuse the ID to update your observation. It expires after five minutes and affects the software robot's costs.

Reports persist atomically in `runtime/live-reports.json`, which is ignored by Git and excluded by the installer. Keys change on restart and are never embedded in the ZIP. Do not put actual field reports, personal locations or access tokens into source control. The local server is for trusted-network collaboration; it is not an internet production service.

9. Phones and physical-device sign-off

Run `python server.py --host 0.0.0.0`, press Enter, and keep that terminal open. In another terminal run `ipconfig`. Phones must use `http://LAPTOP-IP:8000/` on the same trusted Wi-Fi. Phone localhost is not the laptop. Private-network firewall permission may be needed; do not disable the firewall. Guest Wi-Fi may isolate clients; a trusted hotspot can be simpler.

No phone GPS permission is needed for the selected software-robot design. Actual phone GPS would require browser permission and a secure context; an ordinary laptop-IP HTTP address does not provide that. No phone locations are collected automatically.

Test Safari on a physical iPhone and Chrome on Android: pan/pinch, layer switches, ETA, follow mode, source status, portrait/landscape and connection loss. Emulated phone-width browser checks are useful but do not replace those device tests.

10. Test and troubleshoot

From the kit root in a second terminal:

```
python -m unittest discover -s tests -v
```

Press Enter. Test records are fixtures and never presented as actual public observations. For a source diagnosis:

```
python refresh_sources.py
```

Press Enter. Restart after a manual source refresh if an active run needs a clean review. The server otherwise refreshes periodically without external cron or an automation service.

For updated OSM geometry, stop the server and download into a NEW directory with `python refresh_osm.py --out-dir data/osm-next`. Compare that snapshot before changing the active `osm-current` directory. Geometry changes can invalidate saved place/closure identities; the app refuses mismatched reports rather than silently remapping them. Preserve the previous snapshot and reports for review.

Symptom	Action
Old demonstration page	Stop old server; run this kit's server; Ctrl+F5
API 404	A static/old server is probably running
Start disabled	Check source status or unavailable trip; do not relabel stale inputs as live

Symptom	Action
No event heat today	Correct if outside the real event window; inspect source timestamps
No measured crowd display	No people-count sensor is connected; event heat is predicted
No building names in background	Enable optional online OSM tiles; local vector building tooltips still work
Robot jumps back to base	Source change triggered the explicit pause/reset review policy
Phone cannot connect	Check bind address, Wi-Fi, laptop IP and private-network firewall rule
Public source fails	Error is visible; retain cache and use separately labeled simulation for offline exercises

11. MapUI installation and Git instructions

This version targets MapUI. The earlier human-only instructions are preserved only as historical simulation documentation. Do not push this update to human-only by mistake.

From your existing repository terminal inspect first:

```
Set-Location 'C:\Users\noyes\PycharmProjects\MAXX-Map-Submission'
git status --short
git remote -v
git branch --show-current
```

Press Enter after each line. Preserve uncommitted work before changing branches. When the workspace is clean and origin is the expected repository:

```
git fetch origin
git switch MapUI
git pull --ff-only origin MapUI
```

If MapUI is not a local branch yet, use `git switch --track origin/MapUI`. Stop if Git reports conflicts; never force-reset or force-push teammates' changes.

From the EXTRACTED kit's terminal run the optional installer:

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File .\Install.ps1 -Repository
'C:\Users\noyes\PycharmProjects\MAXX-Map-Submission'
```

It verifies MapUI and origin, copies into one NEW `MAXX_MapUI_Live_Kit` directory, refuses an existing destination, and excludes actual runtime reports. It does not edit team backend files or the existing HTML. Run tests from the copied kit, launch it, and review its diff.

From repository root:

```
git add -- MAXX_MapUI_Live_Kit
git diff --cached --stat
git diff --cached -- MAXX_MapUI_Live_Kit/server.py MAXX_MapUI_Live_Kit/world.py
MAXX_MapUI_Live_Kit/public_sources.py
```

Have David and Shek approve the actual staged changes and tests. Only after that review, the submitting teammate can run:

```
git commit -m "Add current-world campus inputs to MapUI software robot"
git push origin HEAD:MapUI
```

The kit does not perform a commit, push, merge, deployment or partner message automatically. Publishing static HTML alone would not run the Python source-refresh backend.

12. Learning and remaining integrations

Follow TEACHING_GUIDE.md to type short exercises into the PyCharm editor and run exact commands in Terminal. CODE_REFERENCE.md contains complete application code. DATA_DICTIONARY.md explains each field. DEMO_CHECKLIST.md records partner evidence.

Remaining real integrations are specifically: physical robot telemetry once hardware exists; full approved class/event feeds; measured crowds; authoritative complete closure data; verified entrances and operational permissions. These are not required to show the software robot consuming the two connected real sources now. They must not be presented as already measured or connected.

Visual update and existing installation

Use the one-file MAXX_MapUI_Visual_Fix.ps1 for an existing kit. It creates a complete local backup under LocalAppData/MAXXMapBackups, updates the kit's code/docs, and preserves `data/` and `runtime/`. It performs no Git writes. Follow PATCH_INSTRUCTIONS.md for the exact command, restart and version endpoint check. Outbound stays purple; return is orange/dashed so both are visible even when they follow identical coordinates.

Learn it by typing: editor versus terminal

The current-world app is supplied in full. Earlier lessons 1-4 teach the core model; they do not send fictitious records to live APIs. First run it, then use these short lessons to understand it. Practice in a **copy** of the extracted kit so an unfinished edit cannot interrupt the demonstration.

Where each kind of code goes

What you see	Where to put it	How to run it
Python: `print(...)`, `def ...`, `import ...`	A `.py` file in the PyCharm editor	Right-click file -> Run, or `python filename.py` in Terminal
PowerShell: `python ...`, `git ...`, `Set-Location ...`	PyCharm's Terminal tab, with PowerShell selected	Paste the command and press Enter
HTML: `<div>`, `<button>`	`web/index.html` in editor	Start Python server, open browser and reload
CSS: `body { ... }`	`web/style.css` in editor	Reload browser; Ctrl+F5 if cached
JavaScript: `fetch(...)`, `L.map(...)`	`web/app.js` in editor	Browser runs it after loading the page
JSON / GeoJSON	The corresponding `.json`/`.geojson` file in editor	Python or JavaScript reads it; do not run it as a program

Do not paste the Markdown backticks or terminal prompts into the editor. Python indentation belongs to the program: preserve it. Every code block in CODE_REFERENCE.md is labeled with its destination file and can be copied without line-number cleanup.

Lesson 1 - Make a Python file

In PyCharm open the kit folder, right-click `lessons` -> New -> Python File. To type from scratch, name your file `my_01_hello.py` so you keep the supplied answer file intact. Type the contents of `lessons/01_hello.py`, which are also in CODE_REFERENCE.md. Save with Ctrl+S.

In Terminal, with the kit directory as the current folder, enter:

```
python lessons/my_01_hello.py
```

Press Enter. Expected output: the project name followed by a dictionary with robot ID and coordinates. Explain it aloud: “This is a record representing a position; it is not moving a real robot.”

If using the supplied file instead, run `python lessons/01_hello.py`. You do not need to install anything.

Lesson 2 - Calculate ETA with units

Type `lessons/02_eta.py` into `lessons/my_02_eta.py`. Run `python lessons/my_02_eta.py` and press Enter. Expected outputs are 100 model seconds and 10 presentation seconds. Change the distance from 120 to 240; both should double. Explain why a 10x clock does not mean the model robot is approved to drive ten times faster.

Lesson 3 - Explain weights and freshness

Type `lessons/03_weighted_point.py` into your own copy. Run it from Terminal. Expected weights: 0.4 initially and 0.2 after an hour. Change only confidence from 0.5 to 1.0; the results become 0.8 and 0.4. Restore your practice copy if needed.

`validate_point` checks required fields and numeric ranges. `timestamp` rejects missing time zones. `effective_weight` applies the time window and freshness. This is the supplied crowd model, not a separate teaching approximation. Dates in this exercise are fixed so its answer remains reproducible.

Explain it: “The weight is an intensity estimate adjusted by trust and age. It is not a measured student count. Distance influence is calculated separately.”

Lesson 4 - Check a real graph route

Study/type `lessons/04_route_check.py`. Run `python lessons/04_route_check.py`. It reads local data, constructs a closure scenario, and checks each adjacent route edge. Expected: `Outbound avoids closure: True`, then the return kind and a distance. The test does not claim the route is physically approved.

Try `robot.configure('2', 'sealed', 1)`. In the full app that scenario gives no trip and disables Start. An `all(...)` check over an empty list is true in Python; therefore a **complete** availability check also requires a nonempty path. The app and automated tests check both. This is a useful reason to test failure states, not just happy paths.

Lesson 5 - Read and explain the full engine

Open `engine.py`. Follow these units in order:

1. `meters` and `edge_key`: coordinate units and stable segment identity.

2. `Campus.__init__`: read team ways, preserve shared node identity, reject unsuitable categories, select connected demonstration stops.

3. `Campus.duration`: sample crowd exposure along each segment and convert meters to seconds once.

4. `Campus.route`: Dijkstra relaxes unblocked edges, then reconstructs the path.

5. `Simulation.configure`: create the selected scenario and plan both legs before movement.

6. `tick`, `action`, `snapshot`: elapsed-time progression, pause/reset and position/ETA response.

You may retype one function at a time into a practice copy. Restore it from CODE_REFERENCE.md if indentation or spelling becomes confusing. Run the test suite after every change to routing or time progression.

Lesson 6 - Understand the server

Open `server.py`. `make_handler` shares the simulation and a lock among requests. GET reads state; POST changes state. The static root is only `web/`, so Python source and operator data are not exposed as web downloads. The familiar map URL is an alias to `web/index.html`; no original map file is replaced.

Terminal commands to run your practice copy are the same as the current-world app:

```
python server.py
```

Press Enter. Leave it running. Open the map in the browser. Use a second terminal for tests or API inspection. Use Ctrl+C in the server terminal to stop it before editing/restarting Python code. Browser reload applies frontend edits; Python edits need a server restart.

Lesson 7 - David's browser code

Read `web/index.html`, then `web/style.css`, then `web/app.js`. The HTML names elements, CSS arranges them, and JavaScript updates their content. `$()` selects an element by ID. `request` handles HTTP and a timeout. `show` updates existing layers instead of rebuilding the map.

Safe exercise: change the introductory paragraph in HTML, save, and reload. Next change a color in CSS. Restore your practice copy afterward. Do not paste those changes into Terminal.

`heat-field.js` calculates the same crowd field as Python. `heat-layer.js` samples it on a canvas. Source and routing-node marker overlays are explanations; turning them off does not change route costs.

Lesson 8 - Shek's telemetry handoff

Read `/api/state` while the software robot moves. Find coordinates, timestamp, mode, phase and ETA. The frontend's polling cadence controls how often it reads positions. The backend clock controls movement. A future real-source adapter should replace the simulator without changing those meanings. Keep `mode: observed` separate from simulated or predicted positions.

Before presenting

Use DEMO_CHECKLIST.md. Both partners should be able to explain one successful route, one blocked route, one unavailable trip and how confidence/freshness affect heat. Run tests in the complete kit, then rehearse using its untouched code.

Lesson 9 - Follow a real source end to end

Type `lessons/05_public_sources.py` into a practice file. Run it with `python lessons/05_public_sources.py` and press Enter. It reads the existing real-source cache and prints source URL, freshness and published records. It does not manufacture data or submit observations.

Then read `public_sources.py`: `refresh` performs actual HTTP GET requests; `parse_projects` and `parse_events` reject unexpected schemas; `snapshot` preserves source age and errors. `world.py` turns an event into a **prediction** rather than labeling it a measured crowd.

Explain it: “The source gives us a real event time and location. We choose a transparent low-confidence crowd heuristic. The robot follows our software route; it is not reporting physical GPS.”

Lesson 10 - Explain failure honestly

Stop the server and start the separate synthetic mode for practice blockages. Do not use the live staff report form to add invented observations. Source-failure tests are in `tests/test_public_sources.py`; run the suite rather than editing the system clock or falsifying the feed. The same map can be tested with software fixtures without calling them real events.

Lesson 11 - MapUI submission

Use the current FULL_GUIDE.md Git commands for MapUI. The historical human-only instructions under simulation_reference are preserved as reference and are not the destination for this update.

Data dictionary

Coordinates and identifiers

Field	Meaning / units	
`coordinates`	`[longitude, latitude]` in decimal degrees, WGS84; Leaflet helpers use reversed order	
`node` / `node_id`	OSM node identity used to connect paths; crossing lines are not automatically connected	
`way_id` / `osm_id`	Source OSM feature identifier	
edge key	Sorted endpoint node IDs joined by `` ` ``; this demo closes a graph segment	
`requested_coordinates`	Original demonstration target, not a verified entrance	
`snap_meters`	Distance from that target to the selected connected graph node	
layer	`walkways`, `buildings`, `delivery` or `restricted`	

Weighted crowd point schema (supplied model)

Field	Type / range	Meaning
`id`, `name`, `source`	Nonempty strings	Record identity, display name, provenance
`kind`	`class`, `event`, `manual`	Source category
`data_mode`	`simulated`, `predicted`, `observed`	Truthful evidence classification
`lat`, `lon`	Finite geographic degrees	Point center
`radius_m`	1-500 meters	Influence reaches zero at this radius
`weight`	0-1	Intensity estimate; this is the field name used for intensity
`confidence`	0-1	Trust factor applied to intensity
`half_life_minutes`	1-10080 minutes	Freshness halves after this age
`starts_at`, `ends_at`	ISO 8601 with zone	Active interval, inclusive start and exclusive end
`updated_at`	ISO 8601 with zone	Source update time, used for freshness
`effective_weight`	Derived 0-1	Intensity x confidence x freshness at planning time

Kernel: `max(0, 1 - distance/radius)^2`. Combine contributions as `1 - product(1 - contribution)` so overlap remains bounded. This is a relative crowd index, not people per square meter or a probability. Sample each walkway at no more than 10-meter spacing; use length-weighted exposure for its time cost. There is no additional routing-node penalty.

/api/state

Field	Meaning
robot_id, mode	SIM-01, simulated; no real hardware dispatch
timestamp	UTC time state was generated, not a GPS observation time
planned_at, revision	Scenario planning timestamp and monotonically increasing configuration version
running	Whether the model clock is advancing
phase	outbound, returning, complete, unavailable
rate	1 or 10; model seconds per wall second
eta_seconds	Remaining demo wall seconds to delivery or base; null when unavailable
eta_travel_seconds	Remaining unscaled model travel seconds
eta_target	delivery outbound, base on return/completion
traveled_meters, total_meters	Actual modeled trip distance, not cost units
outbound_seconds, total_seconds	Full unscaled model travel durations
outbound, return_route	Ordered arrays of longitude/latitude positions
return_kind	exact retrace, rerouted return or unavailable return
blocked	Active outbound closure segments, also considered for return
forecast_return_blocked	Additional segments closed only in the return planning snapshot
crowds, weighted_points	Raw scenario crowd records and planning-time prepared records
weighted_nodes	Node identity, coordinates and score for optional display; never a second routing cost

Units and limits

Base model speed is 1.2 m/s. Maximum crowd cost multiplier is four. These are demonstration assumptions, not legal or campus operating limits. The kit does not enforce a 20-minute service guarantee or blanket 15 mph. There is no delivery dwell. Return closures are explicit scenario snapshots; this is not continuous obstacle detection or a forecast feed.

Current-world mode additions

Field	Meaning
mode	software_robot_with_public_world_inputs; motion remains computed
sources	Per-source URL, successful fetch time, age, freshness and error
projects	Actual published project geometry and phase; not a confirmed closure list

Field	Meaning
`events`	Actual event UID/title/location/start/end/source; not measured attendance
`optional_project_avoidance`	Whether Construction-phase footprints are conservatively excluded
`project_excluded_edges`	Number of graph segments excluded by that optional planning policy
`crowd_note`	Explains predicted event influence and missing people-count/full-schedule feeds
`replan_notice`	Conditions changed; software trip paused/reset for review

Current-world mode fetches sources at startup and every 900 seconds; inputs are stale after 3600 seconds or a failed latest fetch. UI position polling is a separate 1/10-second setting. Geometry refresh is explicit, not a claim of real-time map editing. Crowd timing and routing semantics are in the current guide; the older dictionary fields also describe preserved synthetic/report modes.

Classic visual map additions

`base_place` and each stop contain the named OSM building ID, the original footprint vertex (`requested_coordinates`), the connected path coordinate and snap meters. `access_tags` preserves actual OSM tag keys/values. The restricted-entry marker is a display symbol for access limitations; confirmed dynamic closures remain separate. Return geometry is unchanged by its orange dashed styling.

David and Shek - current-world software robot sign-off

Before the October 27 demonstration

- [ ] Extract kit; open it in PyCharm; run tests.

- [ ] Stop old port-8000 server; start this kit.

- [ ] Confirm projects and events show CONNECTED, with successful fetch times.

- [ ] Explain: robot motion is software; public geometry/projects/event times are actual source data; event crowds are predictions.

- [ ] Pick one shared control operator.

Five-minute rehearsal

1. David: show real paths/buildings, six numbered building targets, Southside Dining base and eight tagged access restrictions and the verified source panel; zoom/pan.

2. Shek: select delivery 2, Apply, 1x or declared 10x, Start; compare 10-second and 1-second reads.

3. David: show ETA, Pause/resume, and view/layer choices surviving refresh.

4. Both: inspect actual published project polygons. Enable optional Construction-area avoidance. Explain why projects are not automatically confirmed sidewalk closures.

5. Both: inspect the real Johnson Center event and its time window. Heat must remain absent outside that window unless there are actual current crowd reports.

6. Shek: explain source-change pause/reset and unavailable-trip behavior. Use the separate simulation mode for reproducible synthetic blockages, never insert fake staff observations into live mode.

Actual observations, if you are on campus

- [ ] Reporter selects their actual location/source; no fake GPS claim.

- [ ] A genuinely observed closure is submitted, timestamped and used in routing.

- [ ] Explicit clearance is submitted only after the obstacle actually clears.

- [ ] A real crowd observation uses a stable ID; updating it does not duplicate it.

- [ ] No personal observation files are staged in Git.

Device evidence

Device / OS / browser	Pan & zoom	Layers	ETA & motion	Portrait/landscape	Tester / result
iPhone / Safari	pending	pending	pending	pending	
Android / Chrome	pending	pending	pending	pending	
Windows browser	pending	pending	pending	n/a	

Defect log

ID	Device/source/scenario	Expected	Actual	Owner	Fix / retest
D-001					

Submission

- [ ] Target branch is MapUI; original map/team backend remain intact.
- [ ] Review staged changes and tests before commit/push.
- [ ] No runtime reports, tokens or private location data staged.
- [ ] Card wording distinguishes connected public inputs, predictions and software movement.
- [ ] Red restricted-entry icon matches map and legend; popups expose actual source tags.
- [ ] Purple outward and orange dashed return are both visible.
- [ ] Layout keeps the team’s heading/instruction/map order, with new controls below.

Verified source register - October 26, 2026

Source	Verification and actual scope	App use
MapUI	Branch exists; checked at commit e88c2cc23ef9ddc495056151b1379eb7b3b1df71	Team baseline/provenance; no remote writes performed
OSM map API	Successful XML response, normalized to 145 buildings and 885 path/restriction features in a central-campus bbox	Real geometry snapshot; not live obstacle sensing
Mason Construction	Official page directly links the following public ArcGIS experience	Authority/discovery chain
Project Map experience	Public app data identifies Project_Polygons layer	Discovery chain to structured data
Project_Polygons service	Public read Query service; schema phase/footprint; 30 public-status polygons returned by the app query during validation, including three Construction-phase records	Actual project overlays; optional conservative avoidance, not asserted confirmed closures
Communication Career Forum	Official event details: Johnson Center, Oct 27, 8:30 AM-12:30 PM Eastern	Published context
Verified event iCalendar	Successful VCALENDAR/VEVENT response; declared America/New_York	Actual schedule/location input; low-confidence predicted crowd effect
Mason scheduling	Documents 25Live and published calendars	Future full approved feed path; not falsely claimed connected

The general Today@Mason Trumba JSON/RSS probes returned errors; they are not active adapters in this kit. No private calendars, student schedules, phone GPS or physical robot telemetry were accessed. Synthetic robot JavaScript in the repository uses software grid coordinates and generated battery values, so it cannot verify a physical robot's location.

Successful fetch metadata is stored under data/public-cache. Failures appear in runtime source status; they do not create fabricated replacement observations. The build verified current public reads; your local network must still be able to reach these sources. Source counts and publisher content can change after this build.

Apply Visual Update 2 to an existing installation

The screenshot showing “Saved Dijkstra route: 1.74 km” and zero buildings/deliveries is the original static map. The new page preserves that map-first appearance and supplies the new functionality. It is served by this kit's Python server.

1. Download `MAXX_MapUI_Visual_Fix.ps1` to Downloads.

2. In the terminal currently running this kit's server, press **Ctrl+C**.

3. At your PyCharm Terminal prompt, paste this ONE command and press Enter:

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File
"$env:USERPROFILE\Downloads\MAXX_MapUI_Visual_Fix.ps1" -Repository
"C:\Users\noyes\PycharmProjects\MAXX-Map-Submission"
```

4. After the patch reports success, paste this ONE command and press Enter:

```
python .\MAXX_MapUI_Live_Kit\server.py
```

5. Keep that terminal running. Open the updated classic map and press Ctrl+F5.

6. Confirm the version endpoint says **MapUI Visual Update 2**.

The canonical path is also available on the correct server: `http://127.0.0.1:8000/html/web/public/dave-campus-map/gis-layers.html`. Use 127.0.0.1 explicitly if localhost is still reaching another server or old IPv6 listener. Do not kill an unknown process. The patch changes no port permissions or firewall settings.

The patch checks MapUI/origin, backs up the prior kit in `LocalAppData\MAXXMapBackups`, then updates code/docs. Existing map data, public-source cache and actual field reports remain local and are preserved. It does not modify the original team map HTML, stage files, commit, push or deploy. If the prior kit contains your edits, they remain in the printed backup path.

Expected: OSM street background, 145 blue building outlines, 885 green path features, six maroon numbered targets, Southside Dining B marker, eight red access-tag restrictions with matching restricted-entry symbols, purple outbound and orange dashed return. Source counts are snapshot-specific and can change after an explicit map refresh.

Weighted heat, routing-node scores and public source connections remain part of this page. Heat does not invent current crowds outside active source windows. Use the bottom-left control to switch advanced layers.

Run `python -m unittest discover -s .\MAXX_MapUI_Live_Kit\tests -v` from repository root after patching. Review the current guide for MapUI staging/pushing; actual runtime reports must stay out of Git.